Undead Legion

Modular Skeleton Army

DOCUMENTATION · UNREAL ENGINE PACKAGE

Unreal Engine 5.4 – 5.8 · Blueprint only

Contents

1. Overview
2. Requirements and setup
3. Folder structure
4. Characters
5. Armor modules
6. Weapons and loadouts
7. Animations
8. The Animation Blueprint
9. Root motion and speeds
10. The character Blueprint API
11. The demo map
12. Using your own animations
13. Technical details
14. Support

1. Overview

Undead Legion is a skeleton army built on **one shared skeleton**. Six class presets, every armor piece and every animation use the same skeleton asset, so any armor piece can be worn by any skeleton and every animation plays on every character.

- **6 skeleton classes**: Knight, Warrior, Archer, Assassin, Mage, Necromancer, each a ready Blueprint Character.
- **48 armor modules** as separate skeletal meshes: helms, hoods, chests, gloves, greaves, boots, skirts, robes and pants.
- **12 weapons** as Blueprints spawned on the hand sockets, with adjustable grips; the recurve bow is skinned.
- **52 animations** at 30 fps: idles, two locomotion sets with root motion, one- and two-handed attacks (four with heavy variants), a held block, a bow shot, casts, specials, hit reactions, staggers, deaths, additive twitches and finger idles. A montage for every one-shot.
- **One Animation Blueprint** with a layer stack: base loop, upper body, left arm, right arm, additive twitches, finger layers.
- **Rigged to the Epic skeleton**: the UE5 mannequin's bone hierarchy, names and bone frames, so any mannequin animation plays on the characters as it plays on the mannequin (section 12).
- A **demo map** with an animation browser and an overview map.

2. Requirements and setup

Unreal Engine 5.4 or later (tested in 5.4; a 5.4 project opens in 5.5 – 5.8). The package is a complete Blueprint project with no C++ and no plugin dependencies. Open `UndeadLegion.uproject` and press Play: the startup map is the animation browser.

Adding the pack to your own project: with both editors closed copy the `Content/UndeadLegion` folder into your project's `Content` folder (or use *Asset Actions > Migrate* on the `UndeadLegion` folder). Everything the pack references lives inside that one folder. The demo's Game Mode is `Demo/Blueprints/BP_DemoGameMode`; set it on the demo map's World Settings if your project overrides the default.

Rendering: the materials are standard lit materials (one master, instances per character and weapon). The project ships with virtual shadow maps and Lumen off and static lighting disabled, which is the setting the demo was tuned for; the assets work under any renderer setting. The eyes are an unlit emissive material and glow through a Bloom post-process volume in the maps.

3. Folder structure

Folder (`Content/UndeadLegion/`)	Contents
`Animations/`	52 animation sequences `A_<Clip>` plus `A_Grip` (finger-pose data); `Montages/AM_<Clip>`, one per one-shot
`Blueprints/`	`BP_UndeadSkeleton` (the base character), `ABP_UndeadSkeleton` (the Animation Blueprint), `Notifies/AN_BowAttach`, `AN_BowRelease`
`Characters/`	`SKEL_UndeadLegion` (the shared skeleton), `PA_UndeadLegion` (physics asset); per class `Skeleton<Class>/`: the body `SK_`, `BP_Skeleton<Class>`, textures, material instances and `Armor/SK_Skeleton<Class>_<Module>`
`Materials/`	`M_UndeadLegion_Master`, `M_Eyes`
`Weapons/`	11 static meshes, the skinned bow `SK_H2Recurvebow` with its skeleton, `Blueprints/BP_Weapon_<Name>`, `Materials/`, `Textures/`
`Demo/`	`Maps/UndeadLegion_Demo` (the browser, startup map), `Maps/UndeadLegion_Overview`, `Blueprints/` (game mode, showcase, orbit pawn, arrow projectile), `UI/` widgets, the floor material
`Documentation.pdf`	this document

4. Characters

Every character shares the skeleton `SKEL_UndeadLegion`: the **UE5 mannequin's hierarchy** (`root`, `pelvis`, `spine_01`–`spine_05`, `neck_01` / `neck_02`, `head`, `clavicle` / `upperarm` / `lowerarm` / `hand` with their twist bones, the metacarpals and fingers, `thigh` / `calf` / `foot` / `ball` with their twist bones, the `ik_hand_*` and `ik_foot_*` bones) with the mannequin's bone frames and our proportions, plus four extra bones: `jaw_01` / `jaw_02` under the head and `pelvis_l` / `pelvis_r` under the pelvis — 93 bones in all. Characters stand 170 cm to the top of the skull with the pelvis at 98 cm, feet on Z 0, facing +Y in the mesh (the Blueprint turns the mesh so the character faces its +X, exactly like the mannequin).

Blueprint	Default armor	Default loadout	Body tris	Armor tris	Total
BP_SkeletonKnight	Helm, Chest, Gloves, Greaves, Boots, Skirt	Sword + shield	7,011	9,717	16,728
BP_SkeletonWarrior	Chest, Gloves, Greaves, Boots, Skirt	Axe + round shield	7,588	9,391	16,979
BP_SkeletonArcher	Helm, Chest, Gloves, Boots, Skirt	Recurve bow	6,364	10,992	17,356
BP_SkeletonAssassin	Hood, Chest, Gloves, Pants, Boots, Skirt	Two daggers	7,588	11,282	18,870
BP_SkeletonMage	Hood, Robe, Gloves, Greaves, Boots	Staff	7,502	10,147	17,649
BP_SkeletonNecromancer	Hood, Robe, Gloves, Greaves, Boots	Staff	7,502	9,882	17,384

Each class is a child of `BP_UndeadSkeleton` (a Character: capsule, Character Movement, the Animation Blueprint, the eyes) and sets only its own data: `DefaultModules` (the armor set above), `StartLoadout` and `DisplayName`. Drop one into a level and it plays its idle; a spawned character is auto-possessed by an AI controller so that Character Movement runs (root motion needs it).

Each character has two material instances, `MI_<Class>_Body` and `MI_<Class>_Armor`, on the master material, with 1024×1024 base color, normal and packed metallic / roughness maps. Armor instances are two-sided, so torn cloth and open hoods read correctly from behind.

Eyes. Two small emissive spheres sit in the eye sockets (sockets `EyeL` / `EyeR` on the head bone). `EyeColor` (HDR, default a glowing red) and `EyesVisible` are variables on the character; `TintEyes(Color)` and `ShowEyes(On)` change them at runtime, for example for an army's team color.

5. Armor modules

Every armor piece is its own skeletal mesh on the shared skeleton, in `Characters/Skeleton<Class>/Armor`. A piece worn by a character is a skeletal mesh component that follows the body as its leader pose, so a Warrior can wear the Knight's helm or the Mage's robe, on any animation.

Set	Modules
Knight	Helm, Chest, Glove_L, Glove_R, Greave_L, Greave_R, Boot_L, Boot_R, Skirt
Warrior	Chest, Glove_L, Glove_R, Greave_L, Greave_R, Boot_L, Boot_R, Skirt
Archer	Helm, Chest, Glove_L, Glove_R, Boot_L, Boot_R, Skirt
Assassin	Helm (hood), Chest, Glove_L, Glove_R, Pants, Boot_L, Boot_R, Skirt
Mage	Helm (hood), Robe, Glove_L, Glove_R, Greave_L, Greave_R, Boot_L, Boot_R
Necromancer	Helm (hood), Robe, Glove_L, Glove_R, Greave_L, Greave_R, Boot_L, Boot_R

Wearing and removing pieces

The functions on the character take the module's **skeletal mesh asset**:

```
WearModule(Module)            attach any module (its material comes with the mesh)
RemoveModule(Module)          take a piece off (a default piece is switched off, a borrowed one
removed)
IsModuleWorn(Module) -> Worn
SetDefaultModulesWorn(Worn)   the character's own set, all on or all off
```

When you swap a piece, take off the character's own piece of the same slot first (for example its `Helm`) so two pieces do not overlap. A module placed in a level on its own is a prop in its rest pose; it animates once it is worn.

6. Weapons and loadouts

Blueprint	Weapon	Tris	Held
BP_Weapon_H1Sword	Sword	824	right hand
BP_Weapon_H1Axe	Axe	1,091	right hand
BP_Weapon_H1Mace	Mace	1,051	right hand

BP_Weapon_H1Dagger	Dagger	897	either hand
BP_Weapon_H1Wand	Wand	1,495	right hand
BP_Weapon_H1HeaterShield	Heater shield	586	left hand
BP_Weapon_H1RoundShield	Round shield	931	left hand
BP_Weapon_H2Longsword	Longsword	881	two-handed
BP_Weapon_H2Axe	Battle axe	1,110	two-handed
BP_Weapon_H2MagicStuff	Staff	1,203	right hand, both hands in the staff cast
BP_Weapon_H2Recurvebow	Recurve bow (skinned, 8 bones)	3,869	left hand
BP_Weapon_Arrow	Arrow	344	right hand, with the bow

Loadouts

`EquipLoadout(Index)` spawns the loadout's weapon Blueprints on the hand sockets and destroys the previous ones (`GetLoadout` returns the current index). The eleven loadouts, right hand first:

Index	Loadout	Right hand	Left hand	Idle that fits
0	Sword + shield	Sword	Heater shield	Idle_Combat
1	Sword	Sword	—	Idle_Combat
2	Axe + round shield	Axe	Round shield	Idle_Combat
3	Mace	Mace	—	Idle_Combat
4	Dagger	Dagger	—	Idle_Combat
5	Two daggers	Dagger	Dagger	Idle_Combat
6	Longsword (2H)	Longsword	on the shaft	Idle_TwoHanded
7	Battle axe (2H)	Battle axe	on the shaft	Idle_TwoHanded
8	Recurve bow	Arrow	Recurve bow	Idle_01
9	Staff	Staff	—	Idle_TwoHanded
10	Wand	Wand	—	Idle_03 / Idle_TwoHanded

Hand sockets and grips

Two transforms meet when a weapon is equipped: the body's socket `RightHandSlot` / `LeftHandSlot` (on `hand_r` / `hand_l`, the same values on all six bodies) and the weapon Blueprint's mesh component offset. Move either one to change how a weapon sits in the hand: the socket in the skeletal mesh editor (copy it to the other five bodies, they share the skeleton), the mesh component in the weapon Blueprint. The socket convention: +Y along the handle toward the blade, +Z out of the back of the hand.

When a hand holds an item its fingers close into the grip pose (from `A_Grip`); an empty hand plays a relaxed finger idle (the finger layers, section 8). Two-handed clips hold the second hand on the shaft themselves: `SetClipHold(Left, Right)` closes a hand that holds nothing while such a clip plays (the demo does this for the two-handed attacks, the two-handed idle and the staff cast).

The bow

`Shoot_01` carries two notifies, `BowAttach` (frame 15, the draw hand reaches the string) and `BowRelease` (frame 21). `BP_Weapon_H2Recurvebow` carries a poseable copy of the skinned bow (`BowPose`) whose bones `Nock` and `Limb_U1/U2`, `Limb_L1/L2` can be driven from the draw hand between the two notifies, and `Demo/Blueprints/BP_ArrowProjectile` is a ready arrow actor (a projectile movement at 15 m/s, no gravity, 4 s life). The demo browser plays the shot as a plain one-shot; the string pull and the fired arrow are left for your own gameplay code, the notifies mark the window.

7. Animations

All clips are 30 fps sequences on `SKEL_UndeadLegion`, `Animations/A_<Clip>`. Loops close seamlessly. Clips marked root motion carry their travel on the root bone; played from a montage they move the character, with `SetRootMotion(false)` they play in place. Every other clip plays in place.

Group	Clips (length in seconds)	Type
Idles	Idle_01 (8.0), Idle_02 (12.0), Idle_03 (10.8)	loop
Combat idle	Idle_Combat (7.33): one-handed weapons and shields, both hands ready	loop
Weapon idles	Idle_TwoHanded (4.67)	loop
Locomotion, heavy set	Walk_Fwd_01 (1.40), Walk_Back_01 (1.97), Run_Fwd_01 (0.97), Strafe_Left_01 (1.47), Strafe_Right_01 (1.47)	loop, root motion
Locomotion, upright set	Walk_Fwd_02 (1.13), Walk_Back_02 (1.47), Run_Fwd_02 (0.93), Strafe_Left_02 (1.30), Strafe_Right_02 (1.30)	loop, root motion
Right arm	Attack_R_01_Stab (1.47), Attack_R_02_Swing (1.97), Attack_R_03_Swing (1.93) and Attack_R_01_Stab_Heavy, Attack_R_02_Swing_Heavy (a bigger lunge of the body)	one-shot
Left arm	Attack_L_01_Stab, Attack_L_02_Swing; Block_L_Idle (6.0, the held shield block)	one-shot; the block loops
Two-handed	Attack_2H_01_Swing (2.50), Attack_2H_02_Swing (1.67), Attack_2H_03_Swing (1.70) and Attack_2H_02_Swing_Heavy (2.00), Attack_2H_03_Swing_Heavy (2.03) (a bigger lunge of the body)	one-shot
Bow	Shoot_01 (2.43: draw and release; notifies BowReset / BowDraw / BowAttach / BowRelease)	one-shot
Magic	Cast_Wand_01 (2.10), Cast_Wand_02 (2.53), Cast_Staff_01 (2.23, both hands on the staff)	one-shot
Specials	Taunt_01 (2.73), Taunt_02 (2.43), Taunt_03 (2.70), Rally (3.03), Cutthroat (3.10), Summon (3.77), AOE_Cast (4.77)	one-shot
Reactions	Hit_01 (0.97, a head-and-shoulders recoil) and Hit_02 (0.97, a gut blow): additive, added over whatever plays; Stagger_01 / Stagger_02 (1.40 each: the hit while stepping back one walk cycle, 66 cm of root motion)	additive overlay / one-shot
Deaths	Death_01 (2.50, drops), Death_02 (2.87, face-down), Death_03 (2.30, on the back), Death_04 (2.90, limp)	one-shot, ends lying
Twitches	Twitch_01 (2.47), Twitch_02 (3.47), Twitch_03 (4.47): head and jaw	additive loop
Finger idles	Hand_Idle_L, Hand_Idle_R (7.97): a relaxed empty hand	loop, fingers only
Grip pose	Grip: the finger pose for a held item (data, not a playable clip)	data

The one-handed attacks and casts carry the whole body, so they read complete when a character stands; on the move they play on the arm and upper-body slots over the legs of a walk or run (section 8). The twitches and the hits are additive against their own first frame. `Animations/Montages/AM_<Clip>` holds a ready montage for every one-shot (29), each in the slot the clip belongs to, for your own Animation Blueprints; the pack's own character creates its montages at runtime from the sequences.

8. The Animation Blueprint

`ABP_UndeadSkeleton` is a flat layer stack rather than a state machine, so every clip can be driven by name from Blueprint and you can build your own state machine on the same sequences. From the bottom up:

Layer	What plays there	Driven by
Base	the looping clips (idles, locomotion, the held block's body) as looping montages in slot `Base`, and full-body one-shots in `DefaultSlot`	`PlayLoop`, `PlayAction` with slot `DefaultSlot`
Upper body	layered per bone from `spine_04`: slot `UpperBody` (wand casts, the shot, Rally over a walk)	`PlayAction`, slot `UpperBody`
Left arm	layered from `clavicle_l`: slot `LeftArm`, and the held block (Block_L_Idle blended in by `BlockAlpha`)	`PlayAction`, slot `LeftArm`; `SetHeldBlock`
Right arm	layered from `clavicle_r`: slot `RightArm`	`PlayAction`, slot `RightArm`
Twitches	three additive loops, Twitch_01..03, each with its own weight	`SetTwitch(Index, Weight)`
Hits	Hit_01 / Hit_02 as additive one-shots over everything	`PlayOverlay` (the demo's Reactions buttons)
Fingers	per hand, layered from the metacarpal and thumb bones: the relaxed finger idle on an empty hand, the grip fist on a full one	automatic on equip; `SetFingerIdle(Left, On)`, `SetFingersSuspended`

A loop and a one-shot play together (the one-shot's montage does not stop the base loop), so a stab on the right-arm slot plays over a walk with the walk's legs and spine, and a shield block held on the left arm stays up through walks and right-hand attacks. Played from standing, the same attack in `DefaultSlot` reads whole, with its step. The demo browser routes each clip this way: a masked clip goes to its slot while a locomotion loop runs and plays full body otherwise.

9. Root motion and speeds

The locomotion loops and the two staggers carry root motion on the root bone; everything else keeps the root still, so hits and deaths displace the body in the pose. Character Movement reads root motion **from montages only** (the character's root-motion mode), which is why the loops are played as montages; a sequence player in the graph would not move the capsule. `SetRootMotion(false)` switches the character to ignore root motion and the loops play in place; the clips then suit an in-place blend space driven by your movement speed, with the measured speeds below. The travel per loop was measured on the exported clips and is identical on all six characters.

Clip	Length	Travel per loop	Speed
Walk_Fwd_01	1.400 s	+72.4 cm	51.7 cm/s
Walk_Back_01	1.967 s	−66.2 cm	33.7 cm/s
Run_Fwd_01	0.967 s	+110.7 cm	114.5 cm/s
Strafe_Left_01	1.467 s	29.3 cm sideways	20.0 cm/s
Strafe_Right_01	1.467 s	29.3 cm sideways	20.0 cm/s
Walk_Fwd_02	1.133 s	+145.5 cm	128.4 cm/s
Walk_Back_02	1.467 s	−113.2 cm	77.2 cm/s

Run_Fwd_02	0.933 s	+175.1 cm	187.7 cm/s
Strafe_Left_02	1.300 s	84.3 cm sideways	64.8 cm/s
Strafe_Right_02	1.300 s	84.3 cm sideways	64.8 cm/s
Stagger_01 / Stagger_02	1.400 s	−66 cm	one-shot

10. The character Blueprint API

All functions live on `BP_UndeadSkeleton` and are callable on every class:

Function	Does
`PlayLoop(Clip)`	plays a looping sequence on the base layer (replaces the current loop with a blend)
`PlayAction(Clip, Slot, BlendIn, BlendOut, Rate) → Length`	plays a one-shot in `DefaultSlot`, `UpperBody`, `LeftArm` or `RightArm` at a play rate; returns the clip's length divided by the rate
`PlayOverlay(Clip)`	plays an additive hit over whatever is playing
`StopActions(BlendOut)`, `PauseActions`	end every one-shot (the loop keeps playing); freeze the pose, for example a corpse
`SetHeldBlock(Held)`	raise or lower the shield block on the left arm, held over anything
`SetTwitch(Index, Weight)`	weight of twitch loop 1..3 (0 off, 1 full)
`SetRootMotion(On)`	root motion drives the capsule, or every clip plays in place
`EquipLoadout(Index)`, `GetLoadout`	the loadouts of section 6
`WearModule`, `RemoveModule`, `IsModuleWorn`, `SetDefaultModulesWorn`	the armor of section 5
`SetClipHold(Left, Right)`	close a hand that holds nothing while a two-handed clip plays
`SetFingerIdle(Left, On)`, `GetFingerIdle`, `SetFingersSuspended(Suspended)`	the relaxed finger idle per hand; suspend both (the demo does so for a death)
`ShowEyes(On)`, `TintEyes(Color)`	the glowing eyes

Variables you may set on an instance or a child class: `DefaultModules`, `StartLoadout`, `DisplayName`, `EyeColor`, `EyesVisible`, `StartActionDelay` / `StartActionGap` (the idle a placed character starts with and how it spaces its first actions).

11. The demo map

`Demo/Maps/UndeadLegion_Demo` is the startup map, an animation browser: pick a class at the top left; the armor panel lists the character's own set as toggles with *All* / *None* and every other class's set to borrow from; the weapons panel the eleven loadouts; the animation list is grouped as in section 7. A masked clip (right arm, left arm, upper body) clicked while a locomotion loop plays goes to its slot over the walk; otherwise it plays full body and the list's idle returns after it. The block, the twitches and the hand idles are toggles. Deaths hold the corpse for two seconds with

the twitches and finger idles off. The footer has Root motion, Turntable, Recenter and Eyes. Drag with either mouse button to orbit, wheel to zoom. `Demo/Maps/UndeadLegion_Overview` shows all six characters side by side.

The browser is `Demo/Blueprints/BP_DemoShowcase` with the widgets in `Demo/UI`; `BP_OrbitPawn` is the camera and `BP_DemoGameMode` ties them together. None of it is needed by the characters.

12. Using your own animations

Mannequin animations. The skeleton is the UE5 mannequin's, so an animation made for the mannequin (Manny / Quinn, the Epic marketplace and Fab packs, your own mocap retargeted to the mannequin) plays on the characters as it plays on the mannequin. Three ways to use them, pick one:

- **Compatible skeletons** (no copies): open your mannequin's skeleton asset (`SK_Mannequin`) and add `SKEL_UndeadLegion` to its *Compatible Skeletons* (or the other way round). Every mannequin sequence and montage then plays on the characters directly — `PlayLoop` / `PlayAction` accept them, and so does any state machine you build.
- **Assign Skeleton**: in a character's skeletal mesh asset choose *Assign Skeleton* → your `SK_Mannequin`. The mesh joins your mannequin skeleton (its four extra bones are added to it) and uses your animations with nothing else to do.
- **IK Retargeter**: with an identical hierarchy the retargeter's chains map automatically; use it when you want baked copies.

The twist bones carry a share of the limb weights, so a mannequin clip twists the forearms and thighs the way it does on the mannequin; the pack's own clips carry their twist on the limb bones.

Your own clips. Import animation FBX files against `Characters/SKEL_UndeadLegion` (set it as the skeleton in the import dialog) and they play on every character and every armor module. Clips you add are played by the same functions: `PlayLoop` for loops, `PlayAction` with a slot for one-shots. To use the pack's clips in your own Animation Blueprint, the montages in `Animations/Montages` are already in the right slots.

Weapon-specific clips expect the matching loadout: the two-handed attacks and idle hold the second hand on the shaft, the bow shot needs the bow in the left hand and the arrow in the right, the staff cast both hands on the staff.

13. Technical details

- **Skeleton**: one asset, the UE5 mannequin's 89 bones (hierarchy, names and bone frames, ik bones included) plus `jaw_01` / `jaw_02` and `pelvis_l` / `pelvis_r`: 93 bones rooted at `root`. Up to 8 bone influences per vertex. Rigged to the Epic skeleton.
- **Scale**: 1 uu = 1 cm, feet on Z 0; the bodies are 169.5 cm tall to the skull, helmets reach about 181 cm.
- **Meshes**: 55 skeletal meshes (6 bodies, 48 modules, the bow) and 11 static meshes, one LOD each, triangulated. Bodies 6,364–7,588 triangles, modules 191–6,582, a dressed character 16,728–18,870, weapons 344–1,495 (the bow 3,869).
- **Materials**: `M_UndeadLegion_Master` with 25 instances (12 character, 12 weapon, 1 demo floor) and `M_Eyes` (unlit emissive). 67 textures: base color (sRGB), normal (DirectX, Y−) and a packed map with metallic in R and roughness in A; 1024^2 for the characters, 512^2 for the weapons, 128^2 for the arrow, every one a power of two.
- **Collision**: the characters collide through their capsule; `PA_UndeadLegion` is the import's physics asset on every body (a capsule per bone, not a tuned ragdoll). The weapon static meshes carry no collision primitives.

- **Root motion**: on the root bone for the loops and staggers; locked root (in place) on everything else. Twitches and hits are additive against their own first frame.
- **Project**: Blueprint only, no C++, no plugins. Startup and default map `UndeadLegion_Demo`; the renderer settings in `Config/DefaultEngine.ini` are the demo's (no static lighting, Lumen and virtual shadow maps off, DX12 SM6).
- **Mobile**: the pack renders under the ES3.1 feature level (Unreal's Mobile Preview): the master material uses default mobile features, every texture is a power of two, and the demo maps carry a sky dome, which the mobile renderer needs to show the SkyAtmosphere. It has not been cooked for or run on a device.

14. Support

Questions, bug reports and requests: through the Fab product page's "Questions" section or the publisher contact there. Please include the engine version and, for an animation issue, the clip and the character.